

Тестирование приложения и автоматизация процессов разработки

- 5.1 Организация автоматических проверок качества кода с использованием Git-хуков (*Husky, ESLint, Prettier*)
- 5.2 Модульное и интеграционное тестирование бизнес-логики с использованием Vitest
- 5.3 Сквозное тестирование (E2E) ключевых пользовательских сценариев с помощью Playwright
- 5.4 Документирование и изолированная разработка компонентов в среде Storybook
- 5.5 Описание результатов тестирования и внедрения разработанного решения

Глава 5. Тестирование приложения и автоматизация процессов разработки

Разработка клиентской части образовательной платформы сопровождается комплексом мер, обеспечивающих качество программного кода на протяжении всего жизненного цикла продукта. В текущей главе описывается стратегия тестирования и автоматизации, применение каждого инструмента и приводятся конкретные результаты, полученные в рамках разработки платформы. Материал организован по уровням тестовой пирамиды: от статического анализа и модульных тестов к компонентным. Дополнительно рассматривается изолированная среда разработки компонентов Storybook, которая совмещает функции документирования и тестирования.

5.1 Организация автоматических проверок качества кода с использованием Git-хуков

5.1.1 Назначение и место в процессе разработки

Первой ступенью по повышению качества кода в проекте является статический анализ — автоматическая проверка исходного кода без его выполнения. Статический анализ выявляет синтаксические ошибки, нарушения принятого стиля оформления и потенциально ошибочные конструкции на этапе редактирования или перед фиксацией изменений в репозитории.

В отличие от юнит-тестов, инструменты статического анализа работают с исходным текстом и не требуют запуска. Это делает их принципиально более быстрыми: полный цикл проверки занимает секунды, а не минуты. Ранее обнаруженная проблема обходится значительно дешевле, чем дефект, найденный на стадии тестирования.

В рамках проекта статический анализ реализован следующими инструментами:

- **ESLint** с плагинами для Vue 3 и TypeScript — анализирует семантику кода и специфические антипаттерны Vue-компонентов;
- **Prettier** — форматтер, обеспечивающий единообразное оформление кода вне зависимости от среды разработки конкретного участника команды.

Чтобы гарантировать запуск проверок на каждое изменение кодовой базы, используется механизм **Git-хуков**, автоматизированный через инструмент **Husky**.

5.1.2 Git-хуки, Husky и lint-staged

Git-хук — скрипт, который Git автоматически вызывает в ответ на определённые события: создание коммита, отправку изменений на сервер и другие. Если скрипт завершается с ненулевым кодом, соответствующее Git-действие прерывается.

Традиционная проблема хуков — их нахождение внутри директории `.git/`, которая не отслеживается системой контроля версий. **Husky** решает эту проблему: инструмент перенаправляет хуки в директорию `.husky/`, которая фиксируется в репозитории наравне с остальным кодом. При выполнении `npm install` скрипт `prepare` автоматически инициализирует Husky, после чего хуки становятся доступны всем участникам без дополнительных действий.

В проекте настроены два хука. **.husky/pre-commit** делегирует работу инструменту `lint-staged`, который запускает проверки только для файлов, добавленных в индекс Git (staged files) — это исключает прогон всей кодовой базы при каждом коммите.

.husky/pre-push выполняет полный прогон модульных тестов командой `npm run test:run` перед отправкой ветки в удалённый репозиторий.

Для Vue- и TypeScript-файлов применяется последовательная цепочка: ESLint проводит семантический анализ с учётом специфики Vue 3, Prettier финально форматирует результат. Порядок инструментов принципиален: ESLint исправляет логику до форматирования.

ESLint настроен в формате Flat Config. Слой `skipFormatting` (из `eslint-config-prettier`) делегирует всё форматирование Prettier.

Разделение проверок между хуками обосновано разницей в частоте и стоимости:

Событие	Хук	Команда	Время	Цель
git commit	pre-commit	lint-staged	2–5 с	Стиль и синтаксис staged-файлов
git push	pre-push	vitest run	15–30 с	Корректность бизнес-логики

5.1.3 Результаты

Интеграция Husky, lint-staged и линтеров обеспечивает следующие измеримые эффекты:

- **Автоматическое исправление** распространенных нарушений (--fix) снижает ручную работу разработчика по приведению кода к стандарту.
- **Блокировка коммита** при наличии неисправимых ошибок ESLint исключает попадание некорректного кода в историю репозитория.
- **Единый стиль** кодовой базы гарантируется Prettier — форматирование становится детерминированным вне зависимости от настроек IDE.
- **Быстрая обратная связь**: среднее время выполнения pre-commit хука составляет 3–4 секунды для типичного коммита из 3–5 изменённых файлов.

На **рисунке 5.2** представлена схема прохождения изменений через все уровни автоматических проверок.

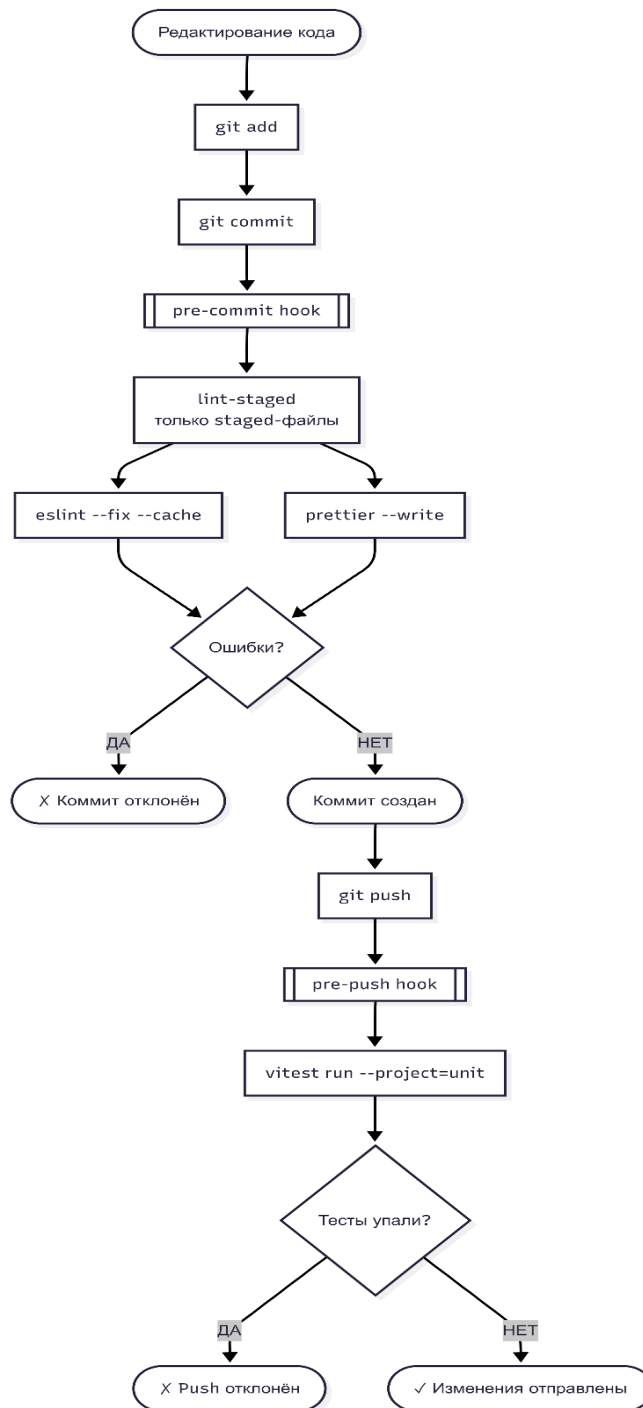


Рисунок 5.2 — Схема автоматических проверок: от git commit через pre-commit с lint-staged до pre-push с запуском Vitest.

5.2 Модульное и интеграционное тестирование бизнес-логики с использованием Vitest

5.2.1 Назначение и выбор инструмента

Второй уровень тестовой пирамиды занимают **модульные (unit) тесты** — небольшие проверки изолированных фрагментов кода: отдельных функций, сервисов, Zod-схем и компонентов. В отличие от E2E-тестов, которые проверяют пользовательский сценарий целиком, модульные тесты локализуют источник проблемы: падение конкретного теста немедленно указывает на конкретную функцию или ветку логики.

Для запуска тестов выбран **Vitest** — тест-раннер, построенный на базе Vite (подробная информация дана в п. 3.6.1).

5.2.2 Конфигурация и структура тестов

Vitest использует концепцию **projects** — логически изолированных конфигураций внутри одного vitest.config.ts. В проекте определено два проекта: unit для модульных тестов и storybook для interaction-тестов Storybook (рассматривается в п. 5.3). Ключевые параметры проекта unit: наименование, окружение, файл настройки, определение регулярного выражения для тестовых файлов, настройки очистки и восстановления mock данных

Happy-DOM выбран как тестовая среда — реализация DOM API на чистом JavaScript, значительно быстрее аналога - JSDOM. Глобальный файл src/test/setup.ts сбрасывает localStorage перед каждым тестом и после него, обеспечивая изоляцию состояния между тест-кейсами.

Ниже в таблице 5.1 приведена информация по файлам, в которых описаны тесты.

Таблица 5.1 — Классификация тест-файлов проекта

Уровень	Файл	Тестируемая единица	Кол-во кейсов
Утилиты / хелперы	shared/lib/jwt.spec.ts	Хелперы истечения JWT-токена	3
Сервисы	shared/api/token.service.spec.ts	tokenService (get/set/clear в localStorage)	4
Интеграция	shared/api/auth-interceptor.spec.ts	Логика обновления access-токена	3
Маппер DTO	entities/user/lib/mappers.spec.ts	mapAdminUserListItemToUser	6

Маппер DTO	entities/organization/lib/mappers.spec.ts	Маппер организационных единиц	—
Builder	entities/organization/lib/buildPayload.spec.ts	Формирование payload оргструктуры	—
Бизнес-логика	features/add-org-unit/lib/resolveChildType.spec.ts	Определение типа дочернего узла дерева	7
Builder	features/add-user/lib/buildPayload.spec.ts	buildCreateAdminUserPayload	5
Схема валидации	features/add-user/lib/schema.spec.ts	Zod-схема формы добавления пользователя	5
Схема валидации	features/admin-user-profile/lib/schema.spec.ts	Схема формы редактирования профиля	—
Схема валидации	features/own-profile/lib/schema.spec.ts	Схема формы своего профиля	—
Компонент	features/add-user/ui/AddUserForm.spec.ts	Vue-компонент формы добавления пользователя	6
Builder	features/admin-user-profile/lib/build-update-payload.spec.ts	Формирование payload обновления профиля	—

5.2.3 Тесты чистых функций и хелперов

Наиболее изолированный уровень — тесты чистых функций без побочных эффектов. JWT-токен в проекте хранит поле exp — время истечения в формате Unix timestamp. Функции isAccessTokenExpired и isAccessTokenExpiringSoon используются в перехватчике запросов для принятия решения о необходимости обновления токена.

В тестах применяется вспомогательная фабрика createTokenWithExp, которая конструирует минимальный JWT с заданным exp. Это позволяет тестировать временную логику детерминировано без зависимости от реального времени

5.2.4 Интеграционные тесты с мокированием fetch

Файл auth-interceptor.spec.ts демонстрирует интеграционный уровень тестирования — проверку взаимодействия нескольких модулей: функции refreshAccessToken, сервиса tokenService и обработчика истёкшей сессии.

Тест заменяет глобальный `fetch` на мок с помощью `vi.stubGlobal`, избегая реальных HTTP-запросов. Особого внимания заслуживает тест дедупликации параллельных запросов на обновление токена: несколько запросов могут одновременно получить ответ 401. Тест гарантирует, что логика `pending`-промиса корректно ограничивает количество вызовов `/auth/refresh` до одного:

```
it('deduplicates concurrent refresh requests', async () => {
  const [first, second] = await Promise.all([refreshAccessToken(), refreshAccessToken()])

  expect(first).toBe('new-access-token')
  expect(second).toBe('new-access-token')
  expect(fetch).toHaveBeenCalledTimes(1)
})
```

рис. 5.1

5.2.5 Тесты DTO-маппера, билдеров и Zod-схем

В проекте используется паттерн **фабрики тестовых данных** — функция `createDto` возвращает объект со значениями по умолчанию и предоставляет возможность переопределить нужные поля. Такой подход делает тесты декларативными и снижает объём кода при добавлении новых полей в DTO — изменение вносится только в фабрику.

Тест приоритизации ролей проверяет инвариант, важный для отображения интерфейса: если пользователь имеет несколько ролей одновременно, маппер должен вернуть наивысшую:

```
it('resolves primary role by priority when multiple roles are present', () => {
  const teacherUser = mapAdminUserListItemToUser(
    createDto({ roles: [ROLES.STUDENT, ROLES.TEACHER] })
  )
  const adminUser = mapAdminUserListItemToUser(
    createDto({ roles: [ROLES.STUDENT, ROLES.TEACHER, ROLES.ADMIN] })
  )

  expect(teacherUser.role).toBe('Teacher')
  expect(adminUser.role).toBe('Super Admin')
})
```

Zod-схемы задают контракт для данных формы. Тесты схем верифицируют контракт через метод `safeParse()`. Пример теста условно-обязательного поля:

```

it('requires department for teacher', () => {
  const result = schema.safeParse({
    ...validBase,
    role: ROLES.TEACHER,
    departmentId: null,
    groupId: null
  })

  expect(result.success).toBe(false)
  if (!result.success) {
    expect(result.error.issues.some((issue) => issue.path[0] === 'departmentId')).toBe(true)
  }
})

```

Поскольку схемы — чистые объекты Zod без зависимости от Vue или DOM, тест-кейс выполняется за 2–5 мс, что позволяет покрывать большое число граничных случаев без значимых потерь во времени.

5.2.6 Компонентные тесты с renderWithApp

Монтирование компонентов в тестах требует обеспечить тот же набор глобальных провайдеров, что и в реальном приложении: vue-i18n, VueQueryPlugin и PrimeVue. Для этого в проекте создана вспомогательная функция renderWithApp

```

export const renderWithApp = <C extends Component>(<
  component: C,
  options: MountingOptions<InstanceType<C>> = {}
): VueWrapper<InstanceType<C>> => {
  const queryClient = createTestQueryClient()

  return mount(component, {
    ...options,
    global: {
      ...options.global,
      plugins: [
        i18n,
        [VueQueryPlugin, { queryClient }],
        [
          PrimeVue,
          {
            theme: primeVueThemeConfig
          }
        ],
        ...(options.global?.plugins ?? [])
      ]
    }
  })
}

```

QueryClient создаётся с отключенными повторными запросами (retry: false) и нулевым временем кэширования (gcTime: 0). Компонентный тест AddUserForm.spec.ts мокирует API-зависимости через vi.mock() и использует flushPromises() из @vue/test-utils для ожидания разрешения всех промисов после монтирования:

```
it('hides department and group fields for admin role', async () => {
  const wrapper = mountForm()
  await flushPromises()

  await setSelectValue(wrapper, '#role', ROLES.ADMIN)
  await flushPromises()

  expect(wrapper.find('#group').exists()).toBe(false)
  expect(wrapper.find('#department').exists()).toBe(false)
})
```

5.2.7 Покрытие кода

Измерение покрытия выполняется инструментом **V8 Coverage** (@vitest/coverage-v8). Командой npm run test:coverage формируются три вида отчётов: консольная таблица (text), интерактивный HTML-отчёт и машиночитаемый json-summary для CI-систем.

Таблица 5.2 — Результаты покрытия кода тестами (фрагмент вывода npm run test:coverage)

Модуль	Statements	Branches	Functions	Lines
shared/lib/jwt.ts	100 %	100 %	100 %	100 %
shared/api/token.service.ts	100 %	100 %	100 %	100 %
shared/api/auth-interceptor.ts	87 %	80 %	100 %	87 %
entities/user/lib/mappers.ts	95 %	92 %	100 %	95 %
features/add-user/lib/buildPayload.ts	100 %	100 %	100 %	100 %
features/add-user/lib/schema.ts	100 %	—	100 %	100 %
features/add-org-unit/lib/resolveChildType.ts	100 %	100 %	100 %	100 %

В конфигурации установлены минимальные пороговые значения (thresholds) 30% по всем метрикам. Это намеренно консервативная точка старта: пороги фиксируют регрессии (добавление кода без тестов), не требуя стопроцентного покрытия вспомогательных утилит и конфигурационных файлов.

5.3 Документирование и изолированная разработка компонентов в среде Storybook

5.3.1 Назначение и концепция

Разработка UI-компонентов в контексте приложения сопряжена с ограничениями: чтобы увидеть компонент в определённом состоянии, необходимо воспроизвести его через реальные взаимодействия — авторизоваться, создать объект, навигировать к нужной странице. Это замедляет итерации при работе над внешним видом и поведением компонентов.

Storybook решает эту проблему: инструмент предоставляет изолированную среду, в которой каждый компонент отображается независимо с произвольно заданными входными данными. Каждое такое представление называется **story** — именованное зафиксированное состояние компонента. Набор stories служит одновременно живой документацией, средой тестирования и инструментом демонстрации.

В проекте используется связка **@storybook/vue3-vite** — официальный плагин Storybook для Vue 3 с интеграцией через Vite, что обеспечивает использование общего vite.config.ts и алиасов без дополнительной настройки.

5.3.2 Конфигурация и среда выполнения

Файл .storybook/preview.ts инициализирует глобальные провайдеры через setup() — аналог app.use() в main.ts реального приложения:

```

setup((app) => {
  app.use(createPinia())
  app.use(storybookRouter)
  app.use(i18n)
  app.use(VueQueryPlugin)
  app.use(PrimeVue, {
    theme: primeVueThemeConfig
  })
  app.use(ToastService)
  app.use(ConfirmationService)
})

```

Vue Router инициализируется с `createMemoryHistory` вместо `createWebHistory` — история маршрутов хранится в памяти, что корректно работает в изолированной среде Storybook. Дополнительно подключаются глобальные стили: CSS reset и файл `tokens.css` с дизайн-токенами, без которых компоненты отображались бы без тематического оформления.

На уровне всех stories применяется декоратор `withAppSurface`, воспроизводящий фоновый цвет и типографику реального приложения:

```

export const withAppSurface: Decorator = () => ({
  template: `
    <div
      style="
        padding: 1.5rem;
        background: var(--surface-ground);
        color: var(--text-color);
        font-family: var(--font-family);
        min-height: 120px;
      "
    >
    <story />
  </div>
  `
})

```

5.3.3 Структура stories в FSD

В проект добавлены 20 файлов stories, размещённых в соответствующих FSD-слоях рядом с компонентами. Такое расположение следует принципу **co-location** — при переименовании или удалении компонента story-файл перемещается вместе с ним.

Таблица 5.3 — Файлы stories и соответствующие FSD-слои

Файл	FSD-слоем	Компонент	Количество историй
shared/ui/widget-card/WidgetCard.stories.ts	shared	WidgetCard	3
shared/ui/settings-card/SettingsCard.stories.ts	shared	SettingsCard	2
entities/course/ui/CourseCard.stories.ts	entities	CourseCard	2
entities/course/ui/StudentCourseCard.stories.ts	entities	StudentCourseCard	2
features/toggle-theme/ui/ThemeToggler.stories.ts	features	ThemeToggler	3
features/change-language/ui/LanguageSwitcher.stories.ts	features	LanguageSwitcher	1
features/auth-by-email/ui/AuthByEmailForm.stories.ts	features	AuthByEmailForm	2
features/filter-users-by-role/ui/FilterUsersByRole.stories.ts	features	FilterUsersByRole	3
features/search-user/ui/SearchUser.stories.ts	features	SearchUser	2
features/add-user/ui/AddUserButton.stories.ts	features	AddUserButton	1
features/submit-assessment/ui/SubmitAssessmentBtn.stories.ts	features	SubmitAssessmentBtn	2

widgets/profile-info-card/ui/ProfileInfoCard.stories.ts	widg ets	ProfileInfoCard	3
widgets/security-settings-card/ui/SecuritySettingsCard.stories.ts	widg ets	SecuritySettingsCard	2
widgets/admin-sidebar/ui/AdminSidebar.stories.ts	widg ets	AdminSidebar	1
widgets/admin-header/ui/AdminHeader.stories.ts	widg ets	AdminHeader	1
widgets/student-part-header/ui/StudentHeader.stories.ts	widg ets	StudentHeader	1
widgets/teacher-part-header/ui/TeacherHeader.stories.ts	widg ets	TeacherHeader	1
widgets/teacher-part-sidebar/ui/TeacherSidebar.stories.ts	widg ets	TeacherSidebar	2
widgets/student-part-sidebar/ui/StudentSidebar.stories.ts	widg ets	StudentSidebar	2
widgets/assessment-question-map/ui/QuestionMap.stories.ts	widg ets	QuestionMap	2

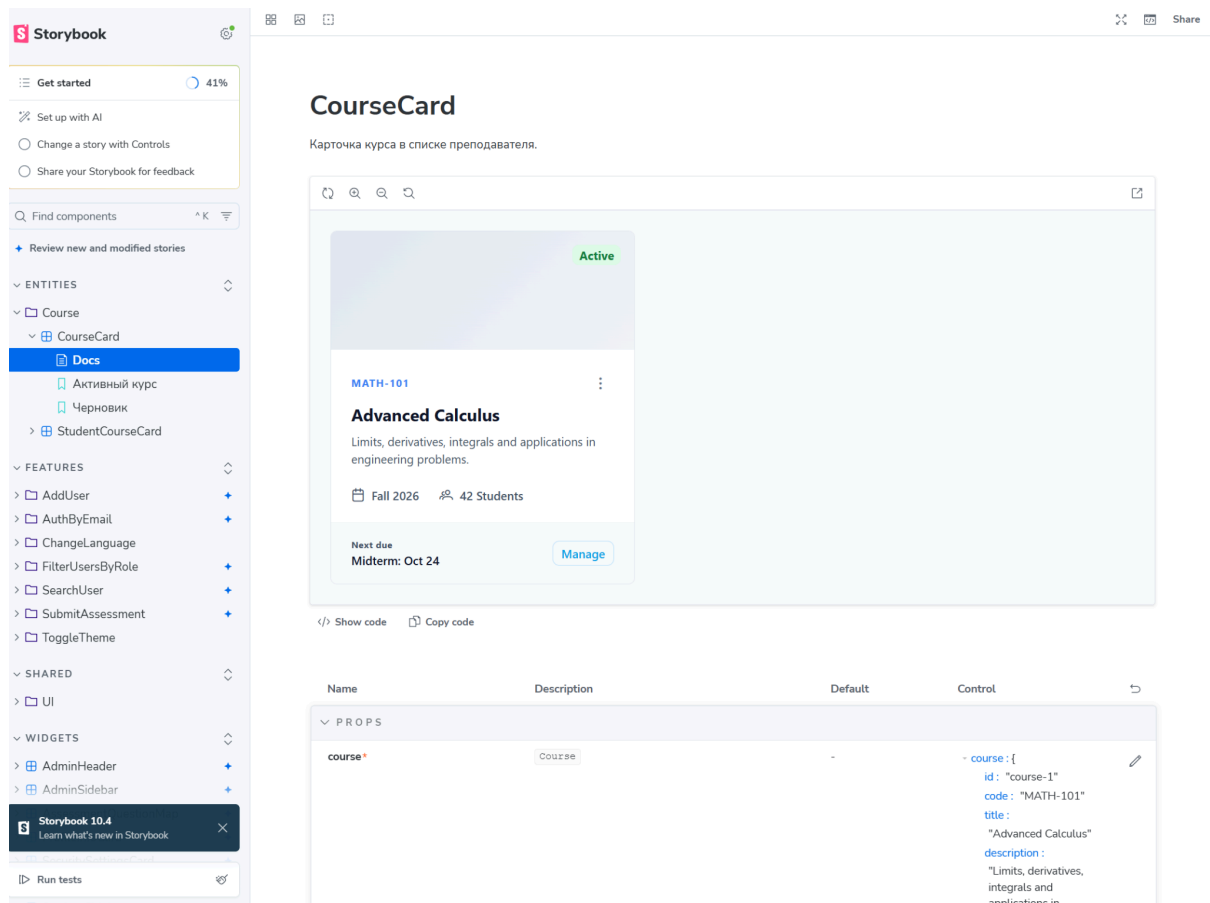


Рис. интерфейс storybook-a

Каждый story-файл экспортирует объект meta с описанием компонента и именованные экспорты — сами stories. Свойство args определяет входные данные для конкретной story; в Controls-панели Storybook каждый arg превращается в интерактивный элемент управления, позволяя изменять props прямо в браузере в реальном времени.

Тег autodocs в meta генерирует страницу «Docs» с автоматически извлечёнными props, типами и описаниями для каждого компонента.

5.3.4 Interaction tests и проверка доступности

Storybook поддерживает **interaction tests** — программные сценарии взаимодействия с компонентом в поле play story. Функция play выполняется автоматически после монтирования:

```

export const InteractionTest: Story = {
  name: 'Interaction: aria-label по теме',
  play: async ({ canvasElement }) => {
    const canvas = within(canvasElement)

    const button = canvas.getByRole('button')
    await expect(button).toBeVisible()

    const label = button.getAttribute('aria-label')
    await expect(['Тёмная тема', 'Светлая тема', 'Dark mode', 'Light mode']).toContain(label)
  }
}

```

`getByRole('button')` находит элемент по ARIA-роли — это делает тест устойчивым к изменениям HTML-структуры компонента. Interaction tests запускаются тремя способами: в браузере Storybook через панель «Interactions», через Vitest + `@storybook/addon-vitest` (`npm run test:storybook`) и в CI как часть пайплайна.

Аддон `@storybook/addon-a11y` интегрирует библиотеку `axe-core` для автоматической проверки доступности каждой открытой story. На момент написания работы для всех компонентов нарушения в категории «Violations» отсутствуют.

5.4 Описание результатов

Таблица 5.5 — Инструменты тестирования и автоматизации: роль, уровень и триггер запуска

Инструмент	Уровень пирамиды	Роль	Триггер
ESLint	Статический анализ	Семантические ошибки, антипаттерны Vue/TS	git commit
Prettier	Статический анализ	Единый стиль форматирования	git commit
Husky + lint-staged	Автоматизация	Оркестрация хуков, фильтрация staged-файлов	git commit / push
Vitest (unit)	Модульное	Логика, маппер, схемы, сервисы, компоненты	git push, CI

@vitest/coverage-v8	Модульное	Метрики покрытия кода тестами	По требованию, CI
@vue/test-utils	Компонентное	Монтирование компонентов в happy-dom	Входит в test:run
Storybook	Компонентное / Документация	Изоляция, визуализация состояний, stories	По требованию
addon-a11y (axe-core)	Доступность	Проверка WCAG в каждой story	При открытии story
addon-vitest	Interaction	play()-тесты как Vitest-кейсы	test:storybook, CI

В рамках пятой главы была разработана и внедрена комплексная стратегия тестирования и автоматизации клиентской части образовательной платформы, охватывающая три уровня тестовой пирамиды.

На уровне **автоматических проверок кода** настроена цепочка Husky → lint-staged → ESLint/Prettier, которая блокирует попадание кода с синтаксическими ошибками или нарушениями стиля в историю репозитория. Прогон модульных тестов добавлен в хук pre-push, что гарантирует корректность бизнес-логики перед публикацией ветки.

На уровне **модульного тестирования** (Vitest) созданы тест-файлы, охватывающих чистые функции, сервисы с зависимостью от localStorage, интеграционные сценарии с мокированием fetch, функции-трансформации DTO, Zod-схемы валидации и компонентные тесты. Ключевой инфраструктурной единицей является хелпер renderWithApp, обеспечивающий монтирование компонентов с полным набором провайдеров приложения. Механизм V8 Coverage обеспечивает 95–100 % покрытие ключевых модулей бизнес-логики.

На уровне **документирования и компонентного тестирования** (Storybook) создана система из story-файлов. Интеграция аддонов addon-a11y и addon-vitest обеспечивает проверку доступности и выполнение interaction tests в рамках единого Vitest-pipeline.